

Doubly Linked List – Length Of Linked List And Time Complexity.

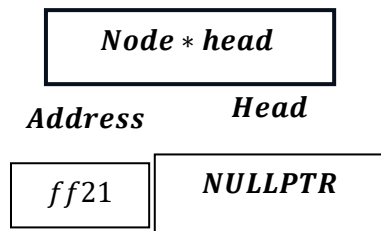
Program :

```
int length()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return 0;
    }
    int count = 0;
    Node *temp = head;
    while (temp != nullptr)
    {
        count++;
        temp = temp->next;
    }
    return count;
}
...
case 9:
cout << "Length of the list is: " << length() << endl;
break;
```

Program Analysis

A. When LIST is Empty

```
if (head == nullptr)
{
    cout << "List is empty." << endl;
    return 0;
}
```

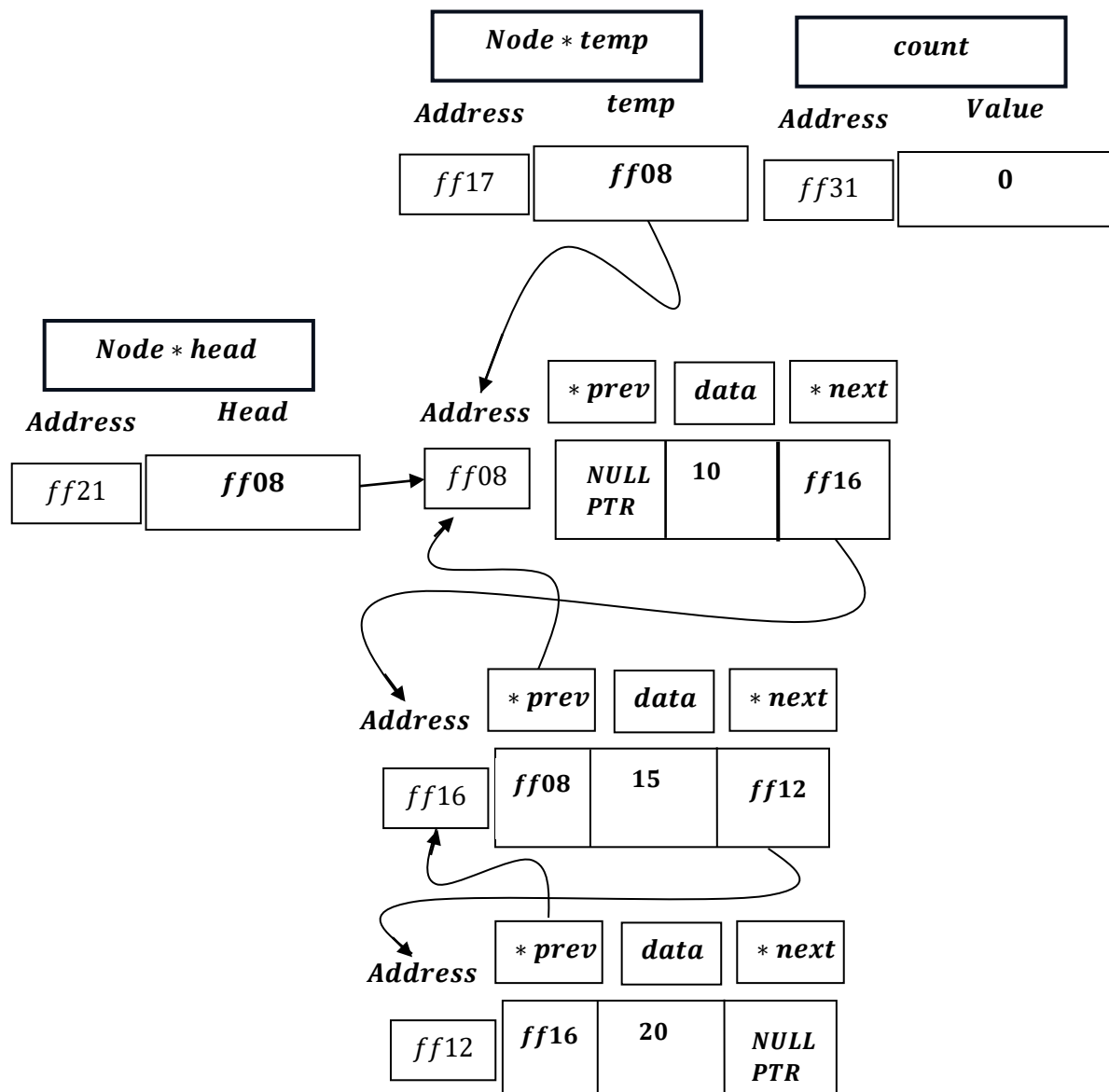


Then, it will print statement : `` List is empty.``

And, return statement will exit from the function.

B. When LIST is not Empty

```
int count = 0;
Node *temp = head;
```



```

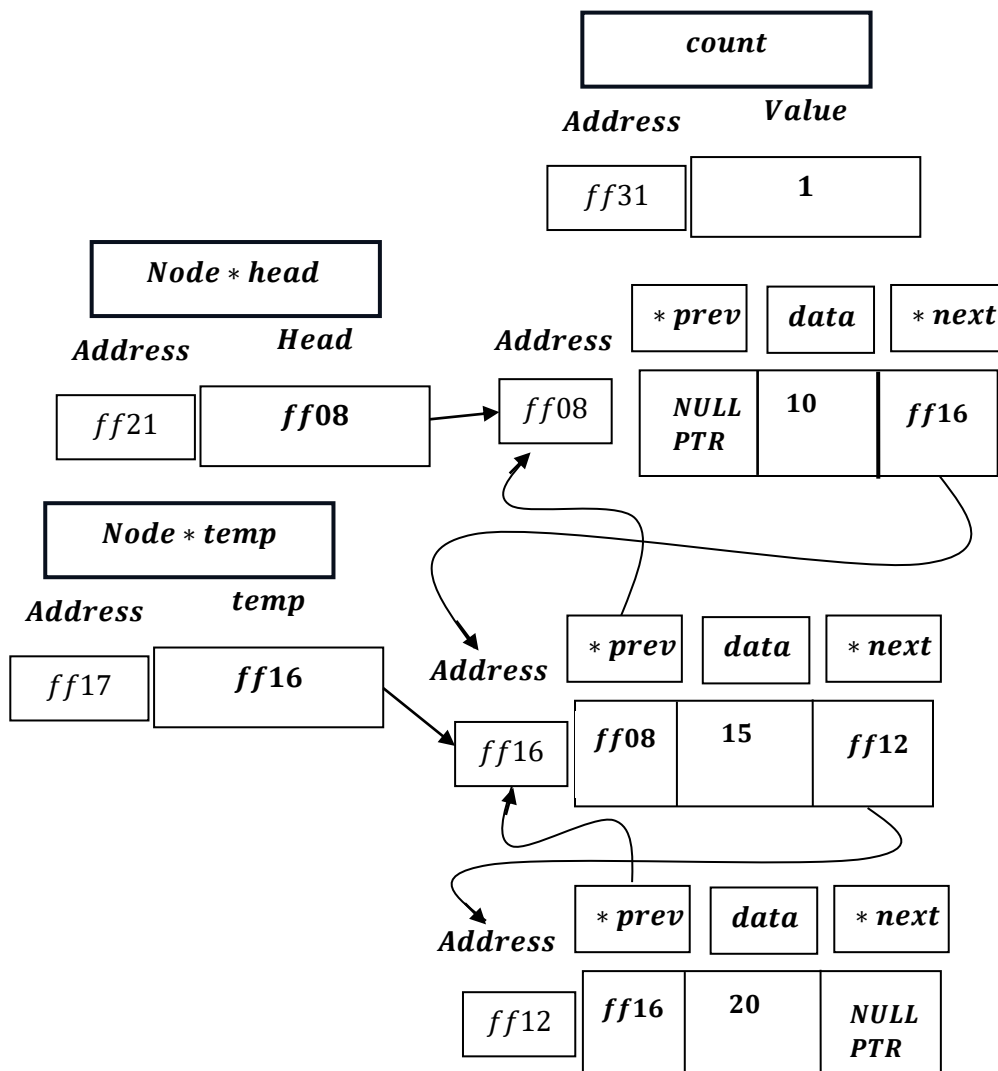
while (temp != nullptr)
{
    count++;
    temp = temp->next;
}

```

Temp $\neq$ *NULLPTR* is true , hence:

count = *count* + 1 = 0 + 1 = 1;

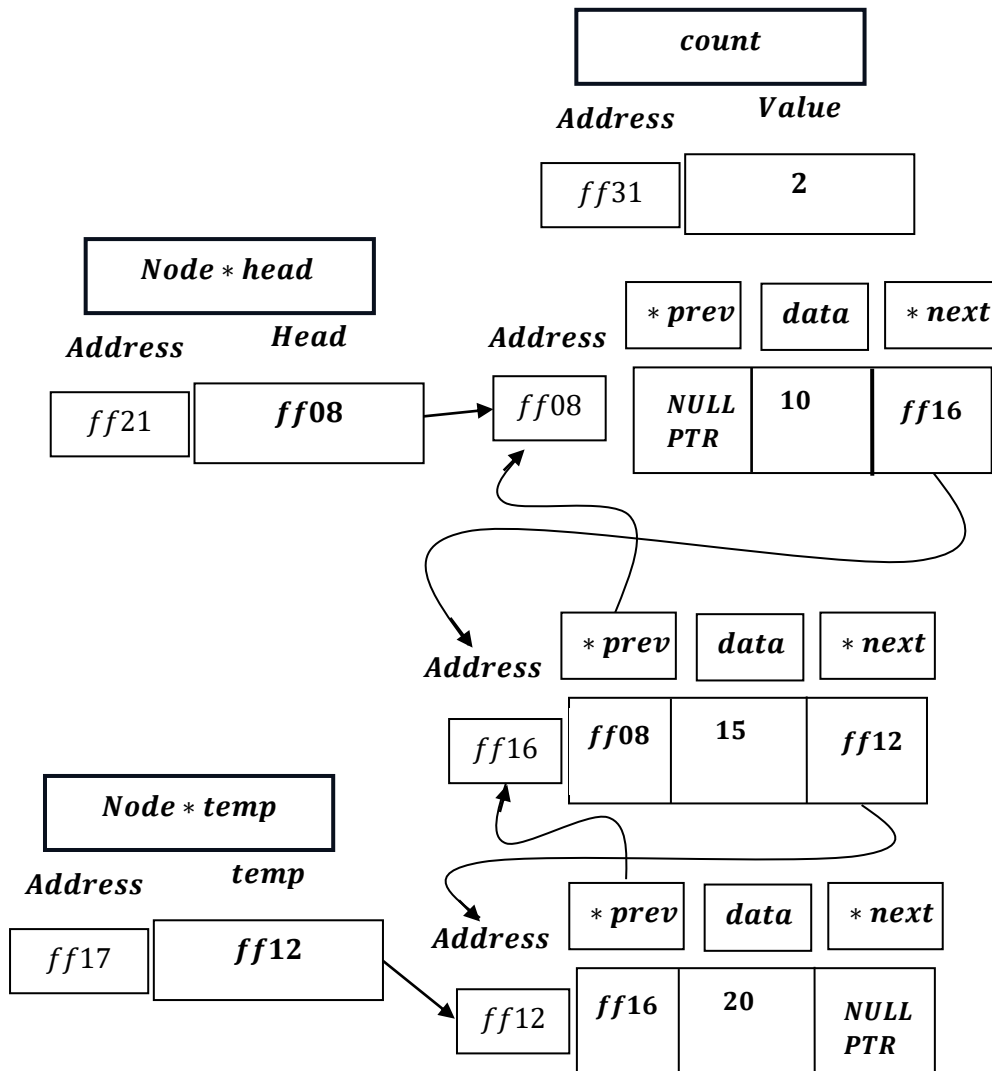
temp = *temp* $\rightarrow$ *next* = *ff16* .



Temp $\neq$ *NULLPTR* is true , hence:

count = *count* + 1 = 1 + 1 = 2;

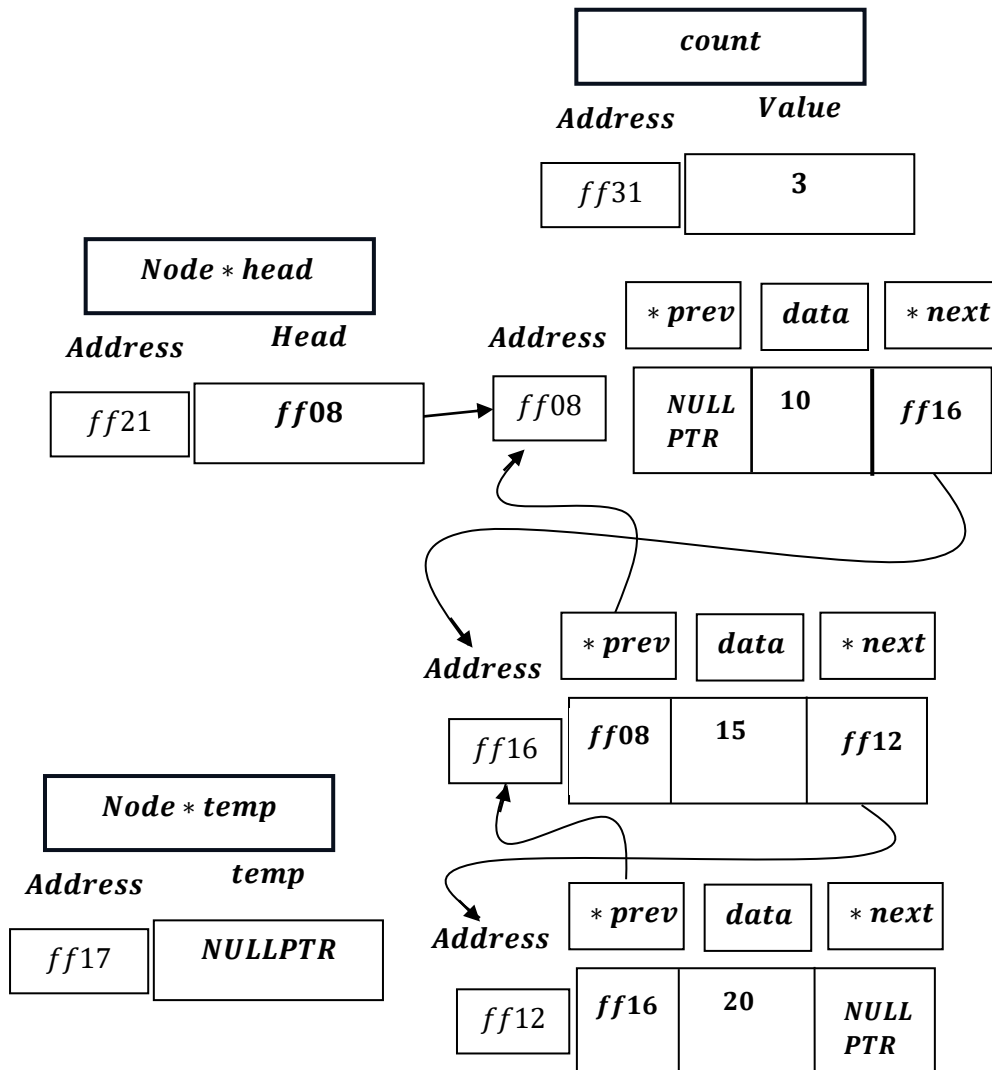
temp = *temp* $\rightarrow$ *next* = *ff12* .



***Temp* $\neq$ NULLPTR is true , hence:**

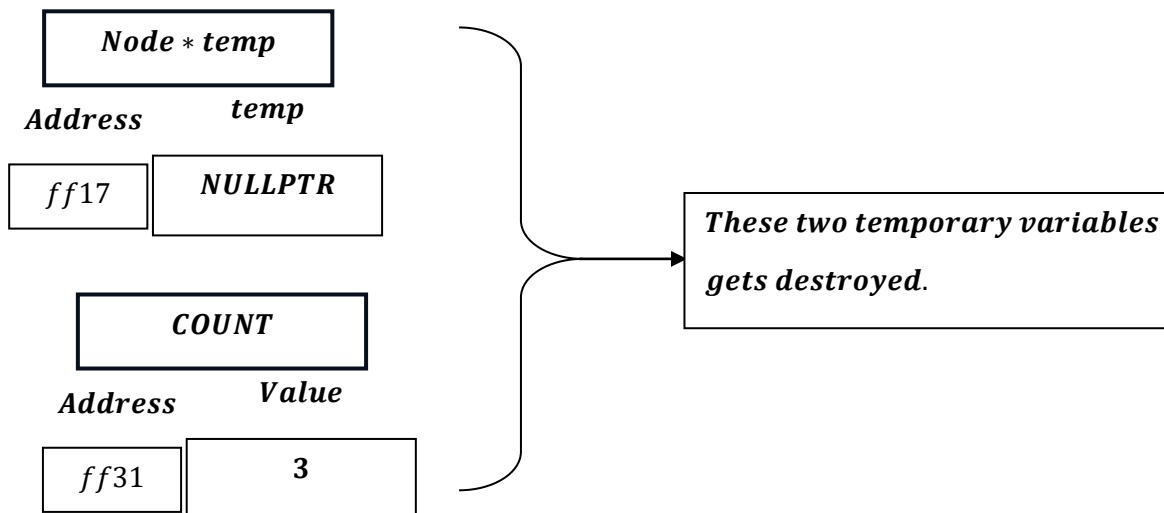
***count* = *count* + 1 = 2 + 1 = 3;**

***temp* = *temp* $\rightarrow$ *next* = NULLPTR .**



***Now, TEMP* = NULLPTR , hence while (*TEMP* $\neq$ NULLPTR) $\rightarrow$ condition fails .**

As function ends ,Temp pointer along with pos variable gets destroyed:



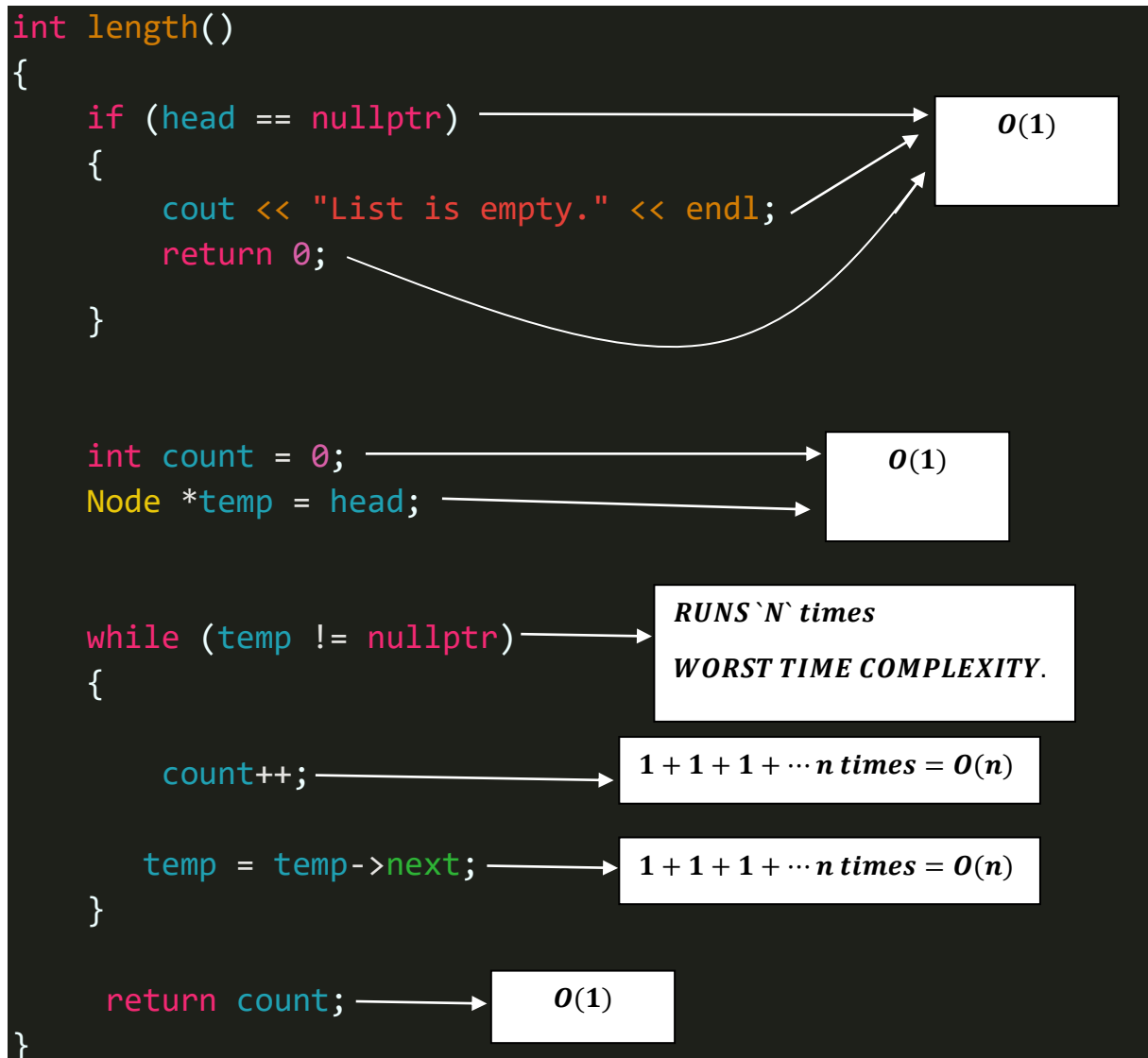
Doubly Linked List – Length Of Linked List [Working Summary]:

This C++ function, `length`, correctly calculates the number of nodes (the length) in a singly linked list.

Working Summary

1. **Handles Empty List:** It first checks if the list is empty by seeing if the `head` pointer is `nullptr`. If it is, it prints "List is empty." and correctly returns 0 as the length.
 2. **Initializes for Traversal:** If the list isn't empty, it initializes a counter variable `count` to 0 and creates a temporary pointer `temp` that starts at the head of the list.
 3. **Iterates and Counts:** The function enters a `while` loop that continues as long as `temp` is not `nullptr` (meaning it hasn't gone past the last node). Inside the loop, it does two things:
 - Increments the `count` by one.
 - Moves the `temp` pointer to the next node in the list (`temp = temp->next`).
 4. **Returns the Total Count:** Once the loop finishes (after visiting every node), the function returns the final value of `count`, which accurately represents the total number of nodes in the list.
-

Time Complexity Of Length Of Linked List Function Module



Program Analysis

Absolute Best Case:

When List is Empty (head = NULLPTR)

→ If `head = nullptr`, this checking takes constant time. The `cout` and `return` statements take constant time. Hence overall process takes, $O(1)$ time.

Worst Case(When List is not Empty)

→ *int count = 0; this assignment takes constant time i.e. $O(1)$.*

→ *Node * temp = * head; this assignment takes constant time i.e. $O(1)$.*

→ *while(temp != nullptr) checking becomes true , then:*

→ *count++ or count = count + 1, increment of length takes, $1 + 1 + \dots n$ times = n ;*

→ *temp = temp → next; such assignment takes, $1 + 1 + \dots n$ times = n ;*

Therefore ,total loop time complexity: $2n$ or $O(n)$.

→ *return count; returns `count = n` , which takes constant time i.e. $O(1)$.*

Hence total time complexity : $O(1) + O(1) + O(n) + O(1) = O(n)$.

Therefore takes $O(n)$ time .

Average Case

We can say , if List is Empty , we get our best case scenario, and when list is not empty , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine list get traversed from beginning to end , hence we can say average case is our worst case here i.e. always the entire list is traversed i.e. $O(n) = \Theta(n)$.

This means the execution time is solely dependent on the length of the list.

If the list is empty, the execution time is constant (best case).

If the list is not empty, the execution time is proportional to the length of the list (worst case).

*There's no element of randomness or varying input data to consider.
Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here.
Hence , lengthOfList function doesn't have a true average case.*

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Length Of List</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case (= Worst Case)</i>	$\Theta(n)$
